



› FOR SOCIETY

SOUND SOURCE LOCALIZATION

TRANSFER DOCUMENT

C. Stijlaart,
T. Verlinde,
T. Spitshoven,
Y. Tama

2024-06-14

VERSION HISTORY

Version	Date	Author	Changes
0.1	12-06-2024	Casey Stijlaart	Doc Setup, Git build/test and recommendation, Teensy all current parts
0.2	14-06-2024	Casey Stijlaart	Finished parts for Teensy and Common

INTRODUCTION

This document serves as a guide for the next team on the current project state.

Various aspects of the project will be discussed regarding to its state, improvements and relevant documents.

TABLE OF CONTENTS

1 Git	1
1.1 Building and Coverage	1
2 Embedded	2
2.1 Common	2
2.1.1 Message Protocol	2
2.1.2 Parser	2
2.2 Teensy	2
2.2.1 UART protocol / Message Handeling	2
2.2.2 Microphone Setup	3
2.2.3 Angle and Decibel Calculation	3
2.2.4 Main Application	3
2.3 ESP32	4
3 Server	5
4 Recommendations	5
4.1 Git	5
4.2 Teensy	5
4.3 Common	5
4.4 Server	5
Glossary	6
References	7
Index	8

1 GIT

1.1 BUILDING AND COVERAGE

As is given, software testing is of great importance, and allowing for automated testing and coverage generation, it will make the process easier. Due to limitations with TeensyDuino, and problems with platformIO, some decisions were made. Due to problems with PlatformIO and GTest, it was decided to make use of CMake. While CMake is a great tool with many different build in tools, TeensyDuino has yet to be supported, which is problem on the TeensyDuino side. This means that for compiling any code, which has to do with TeensyDuino, TeensyDuino functionality must be mocked. Which for the current project state was okay, as only the AudioRecordQueue and the Audio I2SOct was used. Now when it comes to the automation part, a problem arose when it comes to building the custom code coverage target, defined in the CMakeLists. As it worked locally, and on different runners, it did not want to work on the used runners, see Figure 1. Therefore it is recommended to take a closer look into this, as automated testing and coverage is of great importance. Good to mention, only the code coverage has a problem, not the actual building and running of the tests. The following pipeline job gives a clear insight in the problem, <https://projects.fhict.nl/minores/spring-24/sound-source-localization/-/jobs/33671>.

For any further clarification on how to make use of the building, testing and code coverage now, see the README of /Software/embedded/.

```
82 $ cmake --build build
83 [ 33%] Building CXX object CMakeFiles/ParserTest.dir/common/test/parser.cpp.o
84 [ 66%] Building CXX object CMakeFiles/ParserTest.dir/common/src/parser/parser.cpp.o
85 [100%] Linking CXX executable ParserTest
86 [100%] Built target ParserTest
87 $ cd build && ctest --output-on-failure
88 Test project /builds/minores/spring-24/sound-source-localization/software/embedded/build
89 Start 1: parserTest
90 1/1 Test #1: parserTest ..... Passed    0.00 sec
91 100% tests passed, 0 tests failed out of 1
92 Total Test time (real) =  0.01 sec
93 $ make coverage
94 [100%] Built target ParserTest
95 Capturing coverage data from .
96 geninfo cmd: '/usr/bin/geninfo' --output-filename /builds/minores/spring-24/sound-source-localization/software/embedded/report/coverage.info --memory 0'
97 Found gcov version: 13.2.0
98 Using intermediate gcov format
99 Writing temporary data to /tmp/geninfo_datrQq7
100 Scanning . for .gda files ...
101 Found 2 data files in .
102 Processing ./CMakeFiles/ParserTest.dir/common/src/parser/parser.cpp.gda
103 Processing ./CMakeFiles/ParserTest.dir/common/test/parser.cpp.gda
104 geninfo: WARNING: /usr/include/c++/13/bits/stl_iterator_base_types.h:240: unexecuted block on non-branch line with non-zero hit count. Use "geninfo --rc geninfo_unexecuted_blocks=1 to set count to zero.
105 geninfo: ERROR: mismatched end line for _ZN30ParserTest_Initialization_Test8TestBodyEv at /builds/minores/spring-24/sound-source-localization/software/embedded/common/test/parser.cpp:37: 37 -> 40
106 (use "geninfo --ignore-errors mismatch ..." to bypass this error)
107 make[3]: *** [CMakeFiles/coverage.dir/build.make:70: CMakeFiles/coverage] Error 1
108 make[2]: *** [CMakeFiles/Makefile2:145: CMakeFiles/coverage.dir/all] Error 2
109 make[1]: *** [CMakeFiles/Makefile2:152: CMakeFiles/coverage.dir/rule] Error 2
110 make: *** [Makefile:147: coverage] Error 2
111 Cleaning up project directory and file based variables
112 ERROR: Job failed: exit code 1
```

Figure 1: Pipeline Code Coverage Error

2 EMBEDDED

Within the embedded section of the project, it consists of various parts. As team we decided to divide it into three parts, common, teensy and esp32. This as teensy and esp32 both have common implementation, such as the message protocol, but also has target specific implementation.

2.1 COMMON

2.1.1 MESSAGE PROTOCOL

Related Documents : /uart-proto.pdf, system-design-document/system-design-document.pdf

Implementation Placement : ./software/embedded/common/src/communication/*

The message protocol has been completely defined and implemented. This consists of a proper document regarding to the message protocol itself, a message parser and a message constructor. With the help of these aspects, the set up message protocol can easily be used, and if wished extended upon. Meaning that, everything is loose coupled and use the open and closed principle. When new message types must be added, or old ones must be removed, this can easily be done by using the given header source, which contains all the commands, and by ofcourse adjusting the message protocol document. While it has been implemented, due to changes during the implementation, it is recommended to change the uint8_t pointers to uint8_t vectors.

2.1.2 PARSER

Related Documents : /uart-proto.pdf, system-design-document/system-design-document.pdf

Implementation Placement : ./software/embedded/common/src/parser/*

As of today, a parser, to verify the incoming UART messages has been implemented. Yet it is recommended to adapt this into using an uint8_t vector, rather than using a uint8_t pointer as parameter. This to allow for easier usability, as the handling of the data, DataHandler class, uses an uint8_t vector.

2.2 TEENSY

2.2.1 UART PROTOCOL / MESSAGE HANDLING

Related Documents : /uart-proto.pdf, system-design-document/system-design-document.pdf

Implementation Placement : ./software/embedded/teensy/src/data_handler/*

As is given in Section 2.1.1, the message protocol has been defined and implemented. Now when it comes to handling any incoming messages, there has been made use of a class to allow for easily handling incoming data, to relieve the code amount in the main loop, and generally speaking to allow for easy changability and adaptability upon requirement or implementation changes. This class has been defined in the system design document. It is good to mention that the base for this has been implemented and tested. Yet the following must be said. While part of the behaviour has been implemented, some of it has not yet been implemented as this has not yet been implemented on a deeper level. This can easily be seen in the commented out parts in the class definition of DataHandler (located in Software/embedded/teensy/src/data_handler).

When it comes to the UART implementation itself, this has not been implemented as there can be made use of the HardwareSerial library. Although it has been designed, due to time constraints, it was not managed.

2.2.2 MICROPHONE SETUP

Related Documents : /microphone-optimization.pdf, research-documents/microphone-possibilites.pdf, system-design-document/system-design-document.pdf

Implementation Placement : ./software/embedded/teensy/src/audio_manager/*

Note this can only be found when on the given branch later on in the paragraph.

With the help of research, there has been able to be concluded the amount of microphones that could be used with Teensy 4.x, but also what which amount and microphone formation would be the most suitable for the current prototype state and for later iterations. This is all documented in the related documents.

When it comes to the actual implementation, the class AudioManager was defined and created, as is given in /system-design-document.pdf. There has been worked on the implementation, yet it has not been finished due to problems which arose during the setup. The given class and the current implementation can be used to continue with it as it does follow the design. For this, it is good to know what the exact problem is. The problem lies in the actual connection with the microphones and the configuration of it. As can be shown with the unit tests, the rest of the logic in the software works, but the microphones seem to not be configured as expected, and no sound is able to be detected. The current state can be found in the branch /audio-manager-update, see [1].

2.2.3 ANGLE AND DECIBEL CALCULATION

Related Documents : /system-design-document.pdf

Implementation Placement : ./software/embedded/teensy/src/localization/*, ./software/embedded/teensy/src/decibels/*

Note the Localization and Calculator can only be found when on the given branch later on in the paragraph.

The calculation of the angles and decibels is based upon the previous iteration of the project, but had to be reworked as it wasn't up to standard and variable type inconsistencies. While source code was found, upon further inspection, its validity couldn't be verified within the given version of the Teensy Firmware. With the help of software testing, unit tests, parts were verified and were taken directly from the previous iteration, the Decibels class.

When it comes to the rest, it has not been finished. A start was made, but due to hardware problems it was put aside for the time being, but never gotten back to. The finished parts can be found on the main branch, and the unfinished parts are found on the following branch : , see [2].

2.2.4 MAIN APPLICATION

While the main application has yet to be implemented completely, the design and the general flow is there. With the help of the defined classes in the system design document, and the following image, Figure 2, it can be implemented.

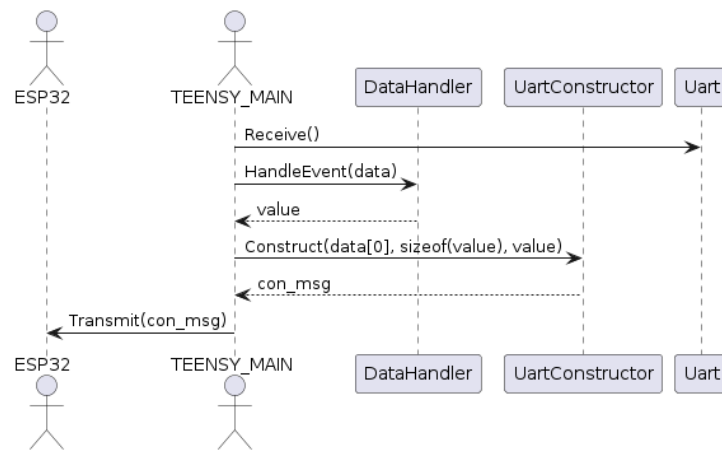


Figure 2: Teensy General Flow

2.3 ESP32

3 SERVER

4 RECOMMENDATIONS

4.1 GIT

- Resolve the problem with the runner and make coverage target for automated testing.

4.2 TEENSY

- **Audio Manager:** Finish the implementation, find root of the microphone problem.
- **Localization (Angle Calculation):** Complete the implementation with the help of live testing, rather than just software testing.

4.3 COMMON

- **Parser:** Make use of an vector rather than an uint8_t pointer.
- **UART:** Follow the given guide to implement the protocol, including the parser.

4.4 SERVER

GLOSSARY

REFERENCES

- [1] "Audio-Manager-Update." [Online]. Available: https://projects.fhict.nl/minores/spring-24/sound-source-localization/-/tree/code/audio-manager-update?ref_type=heads
- [2] "Teensy-Angle-Calc." [Online]. Available: <https://projects.fhict.nl/minores/spring-24/sound-source-localization/-/tree/code/teensy-angle-calc>

INDEX

E

Embedded	6
Embedded-Common	6
Embedded-ESP32	8
Embedded-Teensy	6

G

Git	5
-----	---

S

Server	9
--------	---